



Manual tecnico

1. Objetivo tecnico

Este documento resume como esta construido **Bayes Web Diagnosis** por dentro, para que el equipo lo pueda mantener y extender sin perder tiempo.

2. Arquitectura general

El sistema esta organizado en modulos desacoplados:

- `node.py` : define cada variable aleatoria como nodo de red bayesiana.
- `bayesian_network.py` : administra nodos, orden topologico y consultas de probabilidad local.
- `model.py` : construye la red concreta del dominio web-diagnosis (nodos + CPT).
- `inference.py` : implementa inferencia exacta por enumeracion.
- `scenarios.py` : contiene escenarios de prueba y ejecucion automatizada.
- `utils.py` : utilidades de parseo, formato y salida.
- `main.py` : interfaz de consola y orquestacion del flujo de uso.

Idea general:

Se separa la logica probabilistica, el modelo y la consola para facilitar trabajo paralelo sin bloquear la integracion del equipo.

3. Modelo de datos y Estructuras Base

El corazón de la representación de conocimiento inicial recae sobre dos clases fundamentales:

`Node` y `BayesianNetwork`. Estas clases constituyen la infraestructura técnica sobre la cual actúa posteriormente el motor de inferencia.

3.1 Clase `Node` (`node.py`)

Propósito: Representa de forma individual cada variable aleatoria booleana (nodos de la red) junto con su dependencia condicional respecto a otras variables.

Atributos Internos:

- `name (str)` : El identificador único del nodo (ej. `"Error500"`).
- `parents (List[str])` : Una lista que contiene los nombres de sus nodos padres. Si la lista está vacía `[]` , el nodo actúa como una variable causal.
- `cpt (Dict[Tuple[bool, ...], float])` : La Tabla de Probabilidad Condicional (CPT). Se implementó usando un diccionario donde la llave es una tupla booleana representando los valores de los padres (en el mismo orden en que fueron declarados en `parents`), y el valor es un flotante que indica la probabilidad de que el nodo actual sea `True` .

Reglas de validación internas (`validate_cpt`):

El nodo se autovalida inmediatamente después de su inicialización (`__post_init__` del `dataclass`), garantizando:

- Que el tamaño de la tabla sea exactamente 2^n filas, siendo `n` la cantidad de padres.
- Que la longitud de cada llave coincida con el número de padres.
- Que toda probabilidad asignada se encuentre acotada matemática y lógicamente en el dominio `[0.0, 1.0]` .

Método clave: `get_probability(value, evidence)`

Este método consulta la CPT local. Filtra de la evidencia global únicamente los valores de sus propios padres, construye la tupla de búsqueda y extrae `P(Node=True)` . Si se le solicita `value=False` , calcula silenciosamente el complemento `1.0 - P_true` .

3.2 Clase `BayesianNetwork` (`bayesian_network.py`)

Propósito: Actúa como el Gestor y Contenedor del Grafo Acíclico Dirigido (DAG). Mantiene la coherencia posicional y estructural de los `Node` .

Estructura Interna de Almacenamiento:

- `nodes (Dict[str, Node])` : Diccionario tipo *hash map* para acceder en tiempo constante a cualquier objeto `Node` a partir de su nombre.

- `order (List[str])` : Lista que documenta el orden exacto de inserción de los nodos.

Mantenimiento del Orden Topológico:

Internamente la lógica de la red bayesiana no realiza un "ordenamiento topológico dinámico".

En su lugar, el diseño **obliga a que el registro** de los nodos a través del método

`add_node(self, node)` se haga desde las "Raíces hasta las Hojas" (causas puras primero, síntomas después). Al añadir el nombre del nodo a `self.order` al momento del registro, el arreglo `order` preserva de manera pasiva el orden topológico. Este concepto es vital porque el algoritmo de inferencia por enumeración requiere que cuando analice un nodo hijo, sus variables condicionantes (padres) ya hayan sido trazadas.

Validación Estructural (`validate_network`):

La clase `network` también asume el rol de integridad del DAG comprobando que cada padre de cada nodo declarado dentro del ecosistema exista formalmente en el diccionario `nodes` .

Sin esta revisión preventiva, podrían ocurrir quiebres lógicos de llave faltante en tiempo de inferencia probabilística.

4. Red bayesiana implementada (`model.py`)

Variables causa:

- `BaseDatosCaida`
- `BackendSaturado`
- `GatewayCaido`
- `ConexionInestable`
- `ErrorAutenticacion`

Variables sintoma:

- `Error500` depende de `BaseDatosCaida` , `BackendSaturado`
 - `LatenciaAlta` depende de `GatewayCaido` , `ConexionInestable`
 - `LoginFallido` depende de `ErrorAutenticacion`
 - `ServicioNoDisponible` depende de `GatewayCaido` , `BackendSaturado` , `BaseDatosCaida`
 - `UsuarioAfectado` depende de `Error500` , `LatenciaAlta`
-

5. Motor de inferencia (`inference.py`)

5.1 Funcion `enumeration_ask(query, evidence, network)`

Implementa la consulta posterior:

$P(\text{query} \mid \text{evidence})$

Flujo:

1. Valida que la consulta y la evidencia sean coherentes con la red.
2. Construye distribucion no normalizada para `query=True` y `query=False` .
3. Para cada caso llama `enumerate_all(...)` .
4. Aplica `normalize(...)` .

5.2 Funcion `enumerate_all(variables, evidence, network)`

Implementacion recursiva de enumeracion exacta.

Casos:

- Si no quedan variables: retorna `1.0` .
- Si la primera variable ya esta en evidencia:
 - multiplica su probabilidad local por la recursion del resto.
- Si no esta en evidencia:
 - suma las dos ramas (`True` y `False`) ponderadas por probabilidad local.

5.3 Funcion `normalize(distribution)`

Convierte valores no normalizados a distribucion valida:

$P_i = \text{value}_i / \text{sum}(\text{values})$

Si la suma es `0.0` , lanza `ValueError` para evitar division invalida.

5.4 Validaciones implementadas en esta fase (Integrante 5)

Se agregaron validaciones defensivas para evitar consultas inconsistentes:

- `query` debe existir en la red.
- `query` no puede venir tambien en `evidence`.
- Cada variable de evidencia debe existir en la red.
- Cada valor de evidencia debe ser estrictamente booleano (`True` / `False`).

Estas validaciones se aplican antes de iniciar la enumeracion y generan `ValueError` con mensajes explicitos.

6. Interfaz de consola y validacion (`main.py` + `utils.py`)

6.1 Mejoras implementadas en validacion de entrada

En esta fase se reforzo el flujo de usuario:

- Selecccion de consulta con reintento hasta dato valido.
- Opcion `0` para volver sin ejecutar consulta.
- Lectura de evidencia con validacion estricta por variable.
- Mensajes explicitos para entradas invalidas.

6.2 Convenciones de parseo (`utils.py`)

Entradas verdaderas:

- `s` , `si` , `y` , `yes` , `true` , `1`

Entradas falsas:

- `n` , `no` , `false` , `0`

Entradas desconocidas:

- `d, desconocido, ``, ?`

La funcion `is_valid_yes_no_unknown_input` permite diferenciar valor desconocido de texto invalido.

7. Escenarios de prueba (`scenarios.py`)

La constante `SCENARIOS` define una lista de casos con:

- `name`
- `query`
- `evidence`
- `description`

`run_all_scenarios(network)` ejecuta cada caso y muestra resultados.

Resultados observados en validacion actual:

- `P(BackendSaturado=si | Error500, ServicioNoDisponible, LatenciaAlta) = 0.5801`
 - `P(GatewayCaído=si | ServicioNoDisponible, LatenciaAlta, no Error500) = 0.7320`
-

8. Complejidad y rendimiento

El algoritmo por enumeracion exacta recorre combinaciones de variables ocultas.

- Complejidad aproximada: exponencial en el numero de variables no observadas.
 - Ventaja: implementacion clara, fiel al capitulo 13 y suficiente para redes pequenas (8-12 nodos).
-

9. Como extender el sistema

9.1 Agregar un nodo nuevo

1. Definir nombre, padres y CPT en `model.py` .
2. Asegurar que padres ya existan antes de insertar el nodo.
3. Verificar filas CPT (2^n).
4. Ejecutar programa y validar con opcion `3` .

9.2 Agregar un escenario

1. Crear un nuevo diccionario en `SCENARIOS` .
2. Definir `query` y `evidence` coherentes con la red.
3. Ejecutar opcion `2` para validar.

9.3 Ajustar probabilidades

1. Editar CPT en `model.py` .
2. Re-ejecutar escenarios.
3. Documentar impacto en resultados para trazabilidad.

10. Validacion tecnica realizada

Comandos ejecutados:

```
python -m py_compile main.py node.py bayesian_network.py inference.py model.py scenarios.py utils.  
python main.py
```

Estado:

- Compilacion: exitosa
- Inferencia por escenarios: ejecuta sin errores
- En cada escenario se cumple $P(\text{True}) + P(\text{False}) = 1.0$
- Validaciones de error de consulta/evidencia: validadas

Resultados de referencia en escenarios:

- $P(\text{BaseDatosCaida}=\text{si} \mid \text{Error500}=\text{si}, \text{LatenciaAlta}=\text{si}, \text{LoginFallido}=\text{no}) = 0.2824$
- $P(\text{BackendSaturado}=\text{si} \mid \text{Error500}=\text{si}, \text{ServicioNoDisponible}=\text{si}, \text{LatenciaAlta}=\text{si}) = 0.5801$
- $P(\text{ErrorAutenticacion}=\text{si} \mid \text{LoginFallido}=\text{si}, \text{Error500}=\text{no}, \text{LatenciaAlta}=\text{no}) = 0.4945$
- $P(\text{ConexionInestable}=\text{si} \mid \text{LatenciaAlta}=\text{si}, \text{Error500}=\text{no}, \text{ServicioNoDisponible}=\text{no}) = 0.4573$
- $P(\text{GatewayCaído}=\text{si} \mid \text{ServicioNoDisponible}=\text{si}, \text{LatenciaAlta}=\text{si}, \text{Error500}=\text{no}) = 0.7320$

Comentario:

Con estas pruebas, el motor de inferencia por enumeracion queda estable y listo para ser ajustado por el equipo en fases posteriores (ajuste de CPT, analisis y reporte).

11. Limitaciones conocidas

- No hay aprendizaje automatico de CPT desde datos reales.
- No existe interfaz grafica (solo consola).
- No hay persistencia de consultas/evidencias en archivo.
- Los graficos dependen de `matplotlib` ; si no esta instalado, las opciones de analisis visual no funcionan.

Estas limitaciones son aceptables para el alcance academico del proyecto.